

End-to-End CI/CD Pipeline

Jenkins, Docker, and AWS EC2

Automated full-stack deployment with build, image push, and EC2 release.

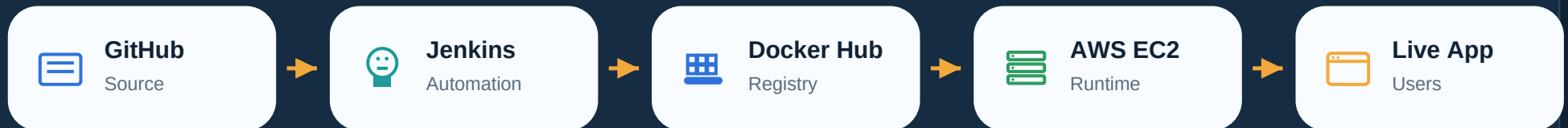
Jenkins Pipeline

Docker Images

GitHub Webhook

AWS EC2 Runtime

Delivery path



Project Snapshot

Bigger readable documentation with a clearer visual story.

OBJECTIVE

Build and automate a complete CI/CD pipeline where a GitHub code push triggers build, Docker image creation, image publishing, and deployment on AWS EC2.

Project overview

React + Vite runs as the client. Node.js + Express runs as the server. Jenkins controls the pipeline, Docker packages both services, Docker Hub stores the images, and EC2 runs the final containers.



Automated

Manual deployment work is removed from the release process.



Containerized

Client and server are packaged as independent runtime images.



Deployed

EC2 hosts the live frontend and backend endpoints.

TECHNOLOGY STACK



Jenkins

Pipeline automation



Docker

Image build and run



AWS EC2

Cloud host



GitHub

Source and webhook



React + Vite

Frontend client



Node + Express

Backend API



Nginx

Frontend hosting

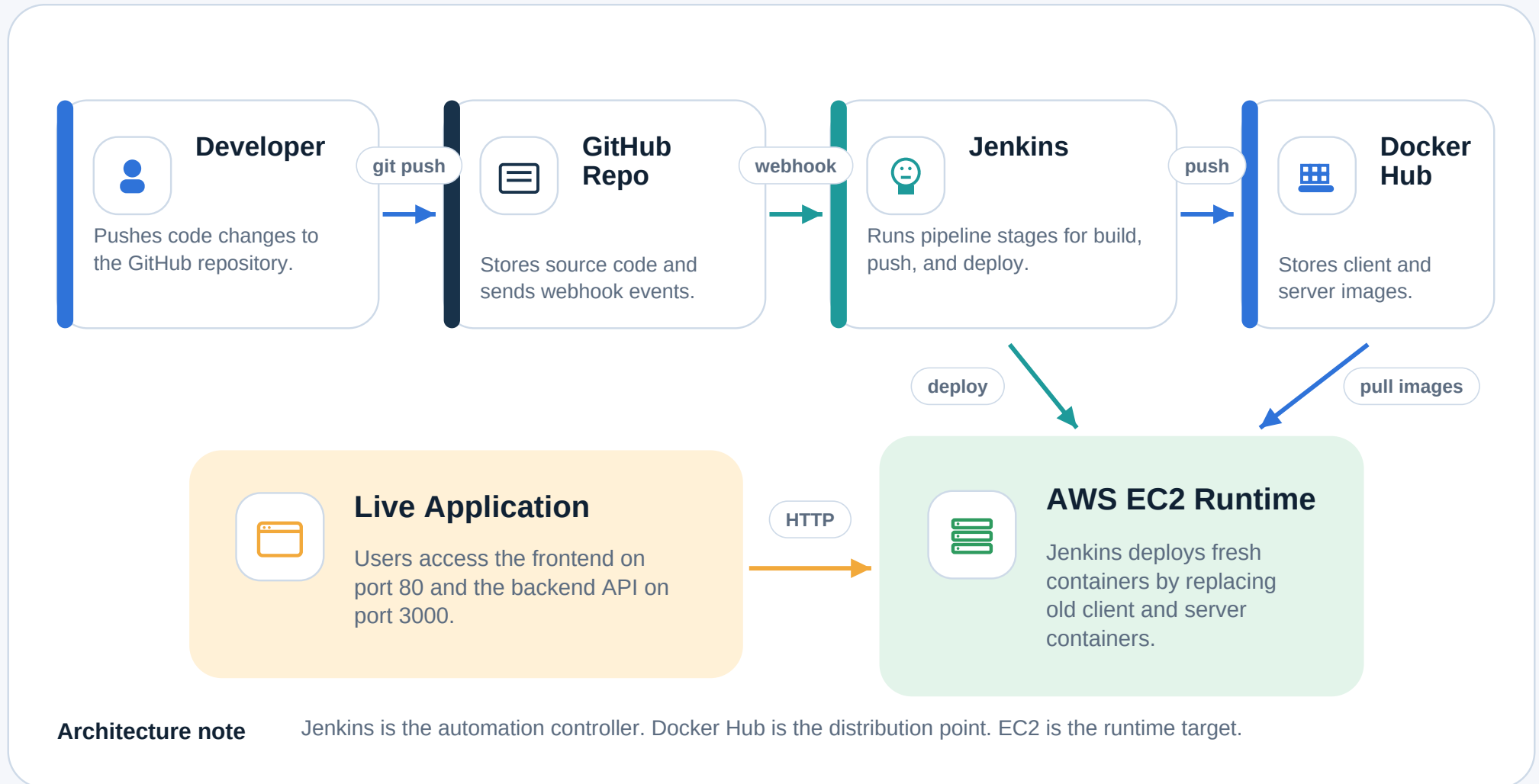


Docker Hub

Image registry

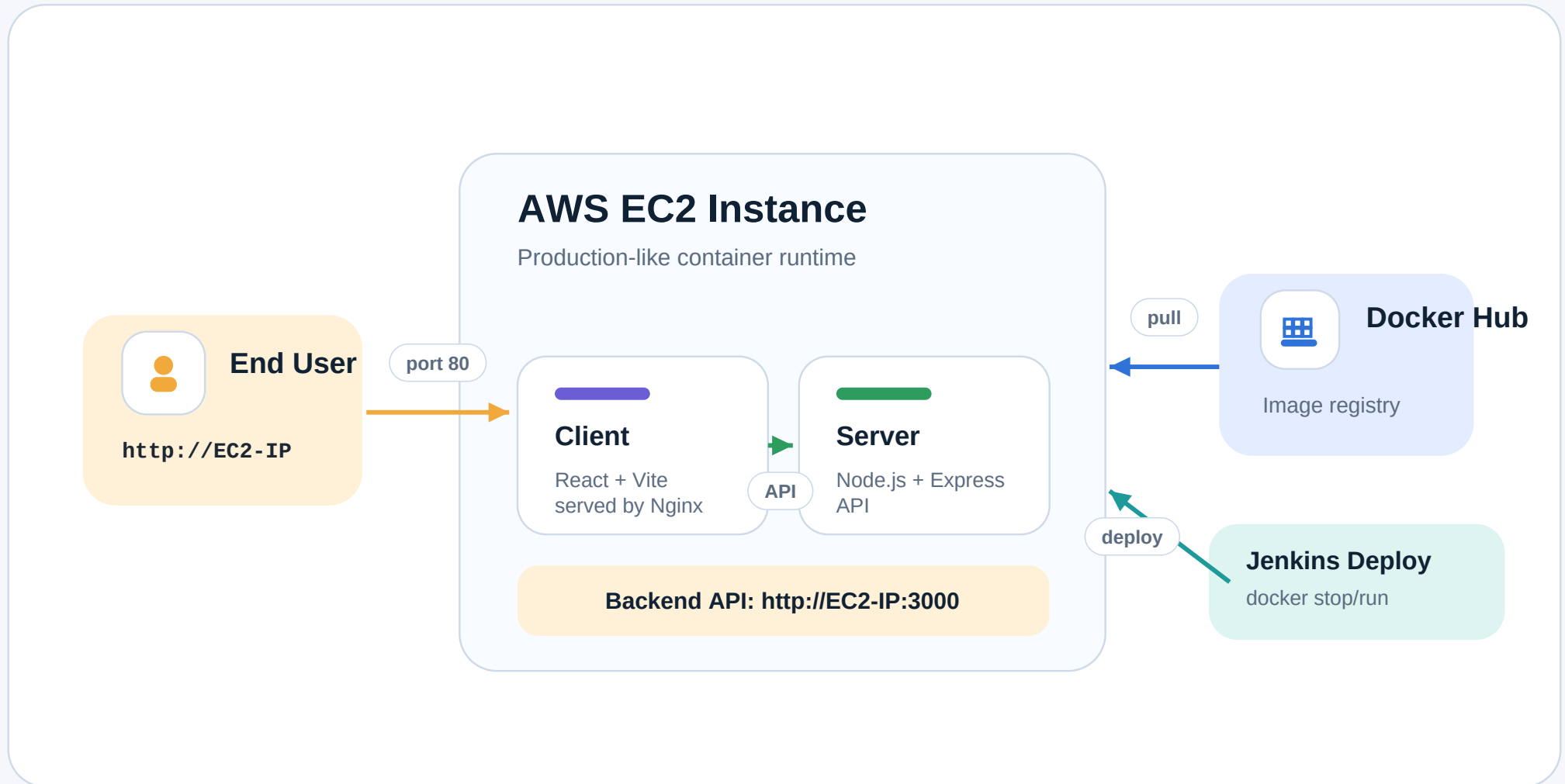
Architecture Diagram

Control plane view from code push to deployable images.



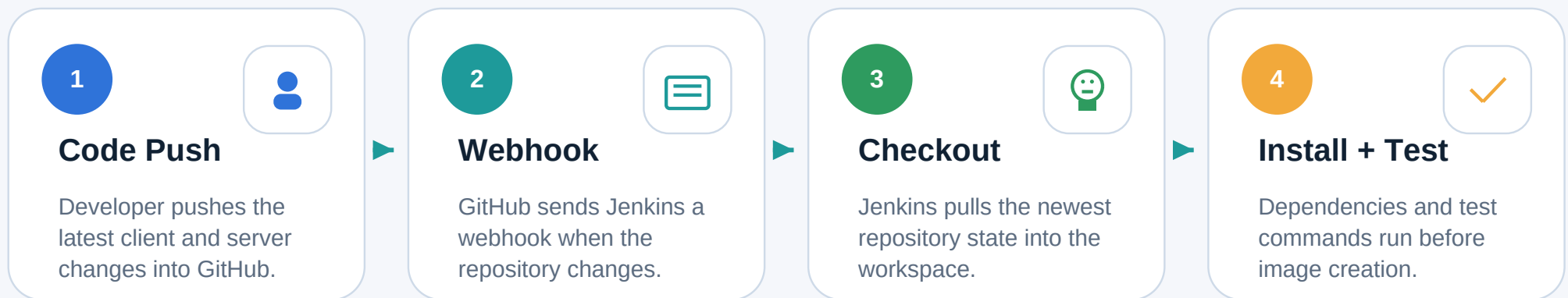
Deployment Topology

A closer look at what runs on the EC2 host.



Pipeline Flow: Trigger and CI

The first half of the delivery flow focuses on source change and validation.

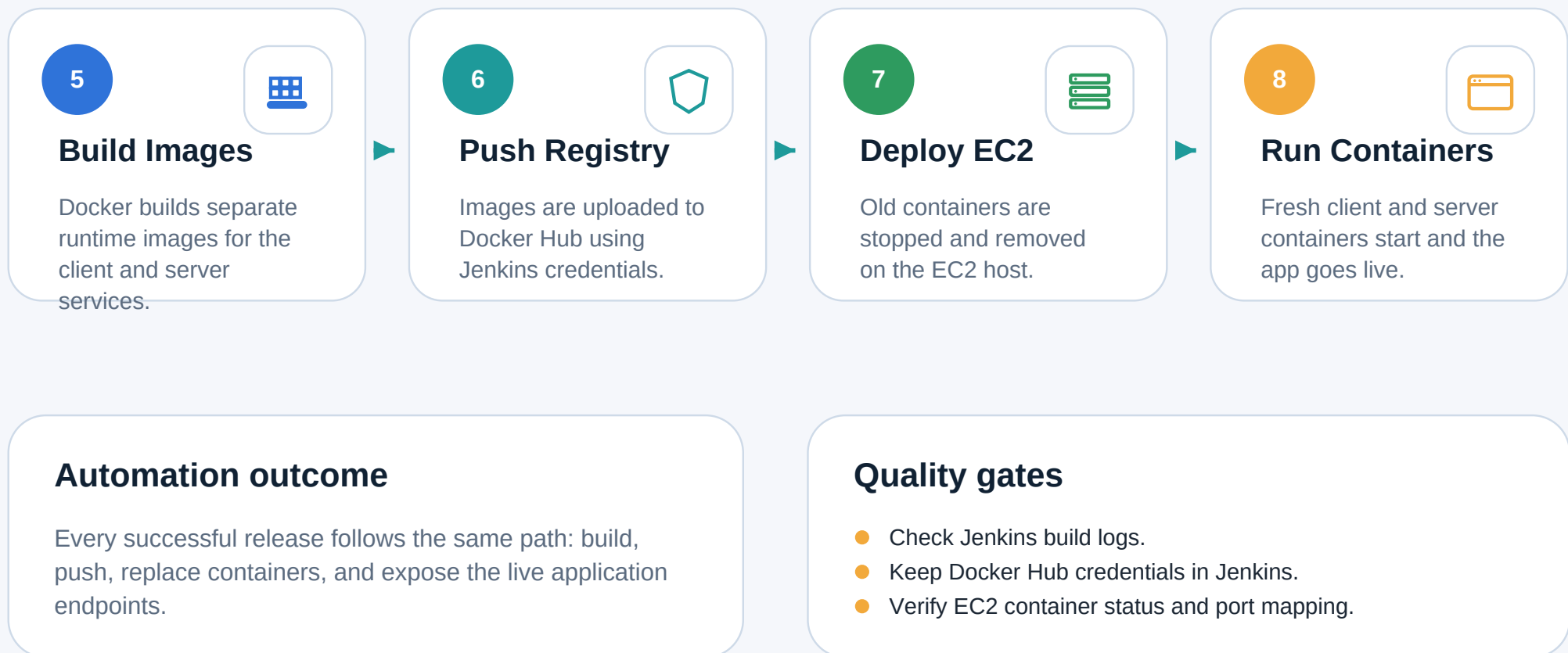


CI purpose

The CI section proves the repository can be pulled, dependencies can be installed, and the application can be prepared before Docker images are created. Problems are caught in Jenkins logs before deployment.

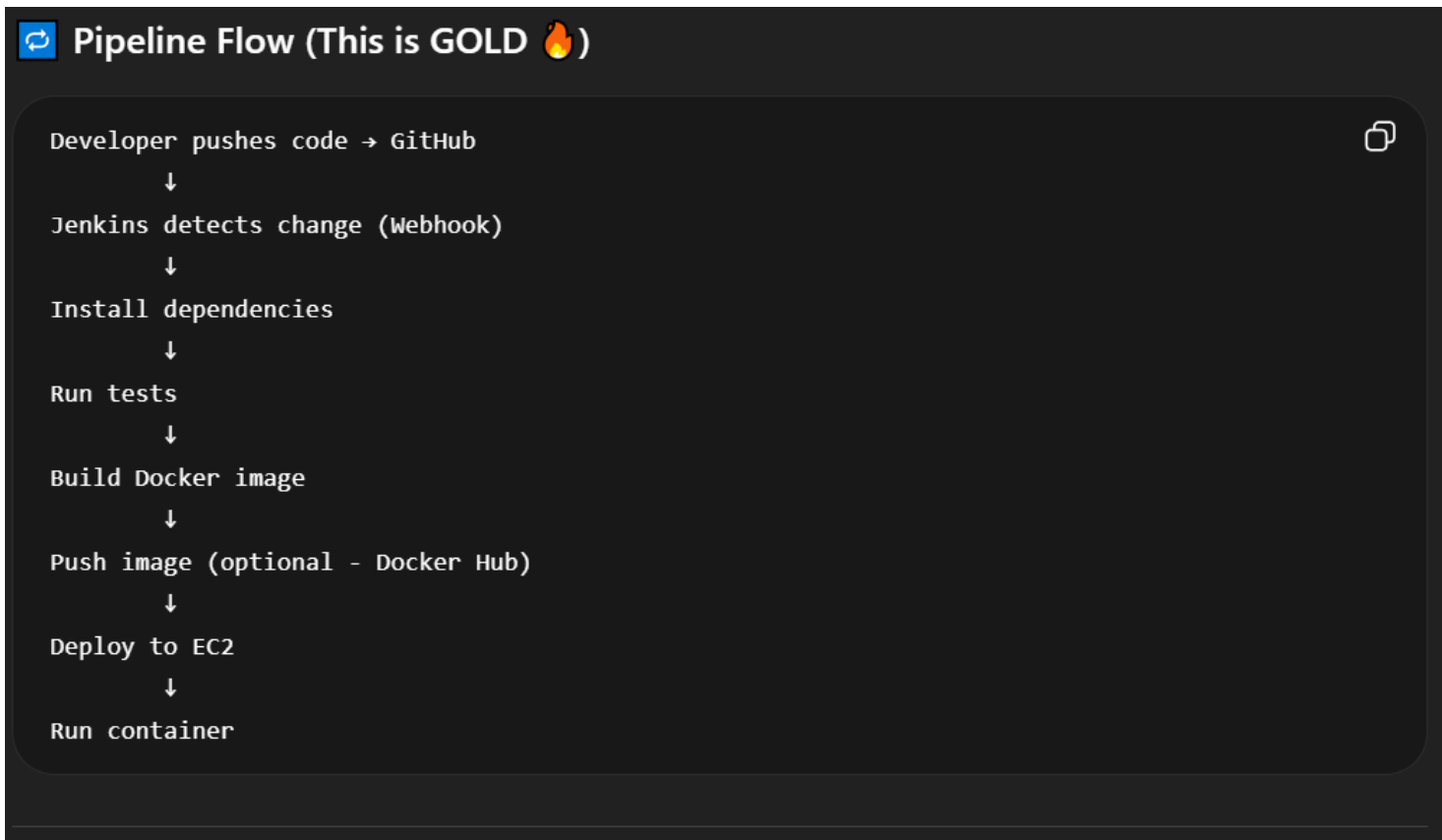
Pipeline Flow: Build and Deploy

The second half packages, publishes, and runs the application.



Pipeline Reference Capture

Original pipeline flow note enlarged for readability.



Original pipeline flow capture

Repository Structure

Service boundaries and important project files.

Folder tree

```
Jenkins-Project/
|-- client/
|   |-- src/App.jsx
|   |-- Dockerfile
|   `-- package.json
|
|-- server/
|   |-- src/app.js
|   |-- Dockerfile
|   |-- .env
|   `-- package.json
|
|-- docker-compose.yml
|-- Jenkinsfile
`-- README.md
```



Client service

React + Vite application packaged into a frontend image and served by Nginx.



Server service

Node.js + Express API packaged into a backend image and exposed on port 3000.

Area	File	Purpose
Client	Dockerfile	Build frontend image.
Server	Dockerfile	Build backend image.
Root	Jenkinsfile	Define CI/CD stages.

Folder Structure Capture

Original folder structure note enlarged for readability.

```
</> Bash

Jenkins-Project/
├─ client/
│   ├─ src/App.jsx
│   ├─ Dockerfile
│   └─ package.json
├─ server/
│   ├─ src/app.js
│   ├─ Dockerfile
│   ├─ .env
│   └─ package.json
├─ docker-compose.yml
├─ Jenkinsfile
└─ README.md
```

Original project folder structure capture

Implementation Setup

Main setup commands for EC2, Docker, Java, and Jenkins.



1. EC2 baseline

Prepare the host and clone the project.

```
sudo yum update -y
sudo yum install git -y
git clone <repository-url>
cd jenkins-project
```



2. Docker runtime

Install Docker and enable it on boot.

```
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ec2-user
```



3. Java + Jenkins

Install Java 21 and start Jenkins.

```
sudo yum install
java-21-amazon-corretto -y
sudo yum install jenkins -y
sudo systemctl daemon-reexec
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Jenkins Configuration

Plugins, credentials, unlock step, and Docker permission.

Plugins installed

- Git Plugin
- Pipeline Plugin
- Docker Pipeline Plugin
- GitHub Integration

Credential stored



dockerhub-creds

Used by Jenkins to log in and push client/server images to Docker Hub.

Unlock Jenkins

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Open: <http://<EC2-IP>:8080>

Docker permission

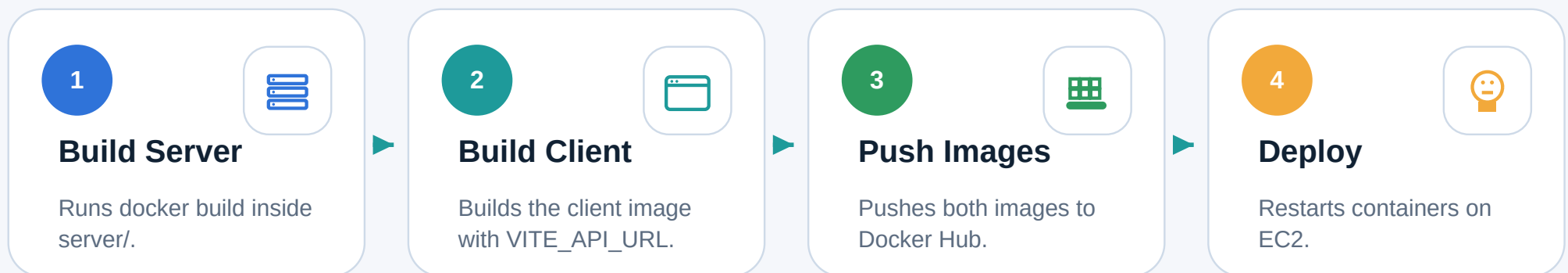
```
sudo usermod -aG docker jenkins
```

Restart Jenkins after changing group membership so jobs can run Docker commands.

Jenkinsfile Design

Pipeline variables and stage responsibilities.

Variable	Example value	Role
CLIENT_IMAGE	satyanarayan4434/jenkins-client	Frontend image pushed to Docker Hub.
SERVER_IMAGE	satyanarayan4434/jenkins-server	Backend image pushed to Docker Hub.
EC2_IP	YOUR_EC2_IP	Used for VITE_API_URL during client build.



Push and Deploy Commands

The deployment logic shown larger and cleaner.

Command flow

```
# Push images with Jenkins credential ID: dockerhub-creds
echo $PASS | docker login -u $USER --password-stdin
docker push $SERVER_IMAGE
docker push $CLIENT_IMAGE

# Replace old containers on EC2
docker stop server || true && docker rm server || true
docker stop client || true && docker rm client || true
docker run -d -p 3000:3000 --name server $SERVER_IMAGE
docker run -d -p 80:80 --name client $CLIENT_IMAGE
```

Frontend

<http://EC2-IP>

Backend

<http://EC2-IP:3000>

Learnings, Challenges, and Roadmap

Final project notes in a more readable format.



Key learnings

- Jenkins pipeline automation
- Docker image build, push, and run lifecycle
- Secure credentials through Jenkins
- Webhook-based deployment



Challenges faced

- Jenkins Java version mismatch
- Docker permission denied for Jenkins
- Wrong frontend API URL
- Webhook not triggering pipeline



Future improvements

- Nginx reverse proxy
- HTTPS with SSL certificate
- Blue-Green deployment
- Kubernetes deployment

Conclusion

This project demonstrates a real-world DevOps pipeline where manual deployment work is reduced, releases become repeatable, and the application stays up to date after source changes.

Acknowledgement

Special thanks to Ulhas Sir and Saroj Sir.